Pontificia Universidad Javeriana · Administración de Bases de Datos · 2026-30

Entrega 2: Integración de los límites de municipios PDET

Potencial solar en techos de los municipios PDET (UPME)

Derek Sarmiento · José Medina · Miguel Duarte
Sebastián Sánchez Vásquez · Samuel Emperador

Prof. Andrés Oswaldo Calderón Romero, Ph.D. · Lunes 9 de noviembre de 2026

Repositorio: `https://github.com/dba-upme-pdet/techos-solares-pdet`

1. Objetivo y flujo

Resultado definitivo. Los reportes publicados por Miguel el 9 de noviembre de 2026 acreditan 170 municipios PDET, 16 subregiones, coincidencia del SHA256 entre preparación y carga y nueve controles espaciales aprobados. La carga y la verificación definitivas corresponden a su entorno; no se atribuyen al computador de Sebastián.

Los resultados de pruebas sintéticas y de la base local vacía se presentan como antecedentes separados. Esta revisión del informe contrasta los archivos de evidencia; no repite la carga ni las pruebas históricas. Las responsabilidades de la tabla indican los tramos asignados. Miguel redactó las secciones de descarga y preparación; los comentarios de sus fuentes conservan que no consta una revisión de Derek y José. La revisión editorial no equivale a una aprobación de esos integrantes.

Esta entrega integra los límites oficiales de los 170 municipios PDET en MongoDB: se descarga la capa municipal del Marco Geoestadístico Nacional (MGN 2025) del DANE, se valida contra la lista oficial de municipios PDET, se convierte a documentos GeoJSON compatibles con el esquema de la Entrega 1, se carga en la colección `municipalities` y se verifica el índice `2dsphere`.

Los scripts comparten un contrato (`scripts/entrega2_contracts.py`, versión `entrega2-mgn-v1`): nombres de campos, CRS, regla de códigos DIVIPOLA y formato de los reportes en `outputs/`. El cruce entre el MGN y la lista PDET es siempre por código DIVIPOLA de 5 dígitos, nunca por nombre.

Tabla 1: Pasos de la Entrega 2, en orden de ejecución.

Script	Responsable	Entrada → salida
`01_download_mgn.py`	Derek Sarmiento	WFS MGN 2025 → capa GeoPackage y `mgn_download_manifest.json`
`01_validate_mgn.py`	Samuel Emperador	capa + `MunicipiosPDET.xlsx` → `mgn_validation_report.json`, resumen y cruce CSV
`02_prepare_municipalities.py`	José Medina	capa + Excel → `municipalities.geojsonl` (170 documentos)
`02_load_municipalities.py`	Miguel Duarte	GeoJSONL → colección `municipalities`
`02_verify_spatial.py`	Sebastián Sánchez	colección → `explain()` sobre `geometry_2dsphere`

1.1. Esquema NoSQL reutilizado

Se conserva el modelo de la Entrega 1 (figura 1). En esta etapa, una subregión agrupa municipios mediante `subregion_id`; cada municipio se guarda como un documento identificado por DIVIPOLA. Las corridas se registran en `ingestion_runs`. La relación espacial municipio–edificación se resolverá en entregas posteriores; aquí no se afirma haber cargado huellas ni calculado potencial solar.

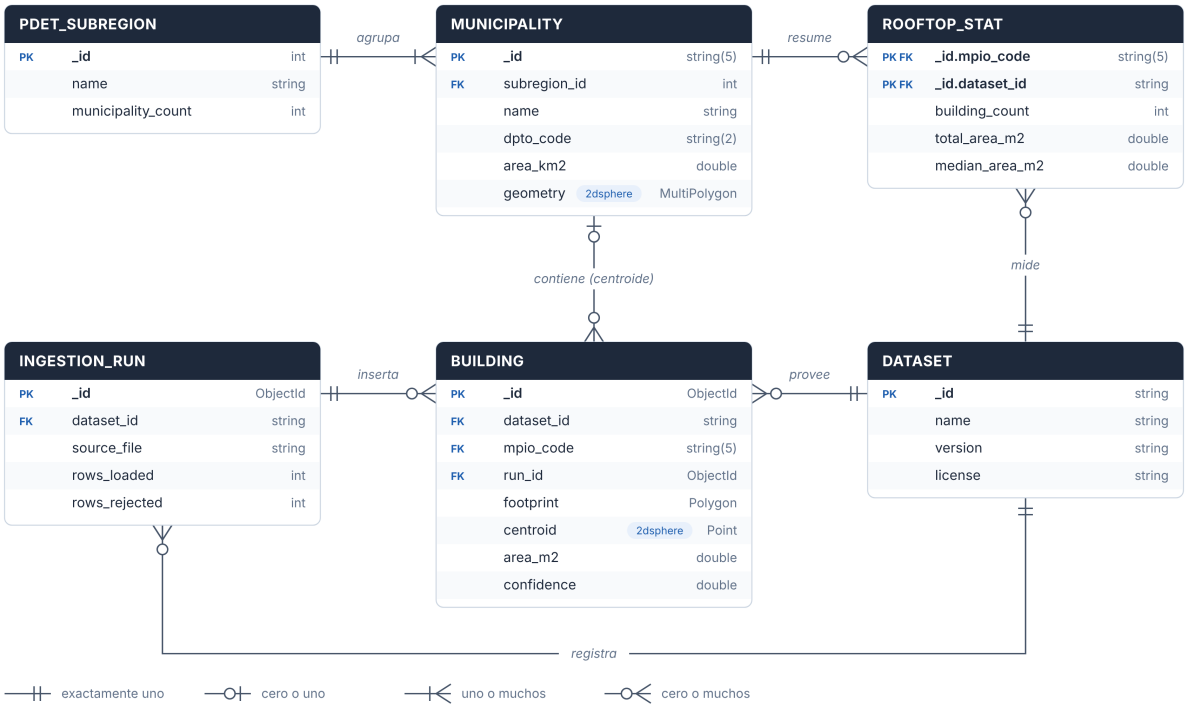


Figura 1: Modelo lógico de referencia conservado de la Entrega 1. Esta entrega materializa los límites municipales, sus subregiones y el registro de carga; las entidades de edificaciones y estadísticas corresponden a etapas posteriores.

Tabla 2: Diccionario resumido de las colecciones utilizadas en la Entrega 2.

Colección	Campos clave y restricciones
`municipalities`	`_id`: string DIVIPOLA de 5 dígitos; `name`, `dpto_code` (2 dígitos), `dpto_name`; `subregion_id`: entero y `subregion_name`; `area_km2`: numérico; `geometry`: GeoJSON Polygon/MultiPolygon en WGS84 con índice `2dsphere`; `source`: objeto de origen y `loaded_at`: fecha BSON.
`pdet_subregions`	`_id`: entero 1–16; `name`: string; `municipality_count`: entero positivo. Los conteos provienen de los mismos documentos municipales.
`ingestion_runs`	`_id`: ObjectId; `dataset_id`, `source_file`; filas leídas, cargadas y rechazadas; fechas de inicio y fin; notas de la corrida.

Los validadores e índices se implementan en `scripts/00_init_db.py`; las referencias entre documentos no equivalen a claves foráneas impuestas por MongoDB. El detalle completo del modelo está en el informe de la Entrega 1.

2. Descarga del Marco Geoestadístico Nacional

Responsable: Derek Sarmiento. Script: `scripts/01_download_mgn.py`.

2.1. Objetivo

Obtener la capa municipal completa del MGN 2025 del DANE (1122 municipios), con los campos que usa el diccionario de datos de la Entrega 1 (`MPIO_CDPMP`, `MPIO_CNMBR`, `MPIO_NAREA`, `DPTO_CCDGO`), dejarla como un GeoPackage legible y acompañarla de un manifiesto que diga de dónde salieron los bytes y si la descarga es nacional y verificable.

2.2. Fuente

La fuente es el servicio WFS 2.0 público del DANE, capa `Serv_CapasMGN_2025:Municipio`, el acceso OGC que publica la ficha `8xdp-fsgm` de `datos.gov.co`. Se pide en GML 3.2 con `srsName=urn:ogc:def:crs:EPSG::4686`, en seis páginas de 200 registros (`startIndex` 0, 200, 400, 600, 800 y 1000). La página renderizada del geoportal y el `FeatureServer` REST no se usan: el cortafuegos del DANE los bloquea para clientes automáticos. La verificación TLS queda siempre activa.

2.3. Entradas y salidas

Tabla 3: Contrato de entrada y salida de `01_download_mgn.py`.

Pieza	Rol	Condición
`config.yaml` (`sources.mgn.wfs`)	entrada	servicio, `typeName`, `srsName`, plan de páginas y `expected_feature_count = 1122`
`outputs/mgn_wfs_source_receipt.json`	entrada	por página: URL, bytes, `sha256`, `content_sha256` y `server_timestamp`
`data/raw/mgn2025/wfs/page_XXXX.gml`	salida	respuestas del WFS guardadas byte a byte (fuera de Git)
`data/raw/mgn2025/wfs/municipio.gpkg`	salida	1122 municipios, EPSG:4686, longitud/latitud (fuera de Git)
`outputs/mgn_download_manifest.json`	salida	`status = complete`, `claims_national_download = true`, SHA256 de la capa y coincidencia de cada página

2.4. Método

1. **Plan oficial.** El recibo solo cuenta si su servicio, `typeName`, `srsName`, URLs, `startIndex` y `number_matched` son los de `config.yaml`. Un recibo de otro tamaño, aunque sus bytes coincidan, no habilita una descarga nacional.
2. **Caché o descarga.** Si las seis páginas locales coinciden con el recibo, no se contacta el servidor. Si no, se hace un intento por página y la corrida se detiene al primer fallo de red o TLS. Una respuesta que no coincide con el recibo se rechaza.
3. **Coincidencia por bytes o por contenido.** El WFS escribe en el elemento raíz un atributo `timeStamp` con la hora de cada respuesta, de modo que un cambio de ese atributo altera el SHA256 binario aunque el contenido geográfico sea igual. Por eso el recibo guarda además `content_sha256`: el SHA256 de la página con el valor de ese único atributo vaciado. Una página vale si coincide por bytes o por contenido; los archivos descargados no se modifican.

4. **Lectura del GML.** Los elementos vienen con alias en español y se mapean de forma explícita (`Código_DANE_concatenado_departamento_y_municipio` → `MPIO_CDPMP`, `Nombre_municipio` → `MPIO_CNMBR`, `Área_departamento_en_kilómetros_cuadrados` → `MPIO_NAREA`, entre otros). Cada `posList` viene en latitud/longitud, como declara EPSG:4686, y se guarda en longitud/latitud.
5. **Controles de la capa.** Exactamente 1122 registros, igual a `numberMatched`; códigos de 5 dígitos sin repetidos; un único `srsName` en todas las páginas.
6. **Unidad del área.** El nombre del elemento dice kilómetros cuadrados; además se compara `MPIO_NAREA` con el área geodésica WGS84 de cada contorno. Si la mediana del cociente no cae entre 0,5 y 2, no se declara la unidad y la descarga no queda como nacional.
7. **Manifiesto honesto.** Si algo falla, el manifiesto actual queda en `blocked` o `invalid` con `claims_national_download = false`, y un manifiesto exitoso anterior se conserva aparte en `mgn_download_manifest.history.json`. Si la capa y los hash no cambiaron, la siguiente corrida solo la revalida.

2.5. Pruebas

`python scripts/01_download_mgn.py -self-check` usa fixtures etiquetados y nunca cuenta como descarga del DANE. Resultado: **22/22 OK**. Cubre la elección del ZIP correcto en el HTML del geoportal, la lectura de la unidad desde metadatos, el rechazo de CRS ausente o distinto, de capas sin `MPIO_NAREA` o ilegibles, la inversión de ejes del GML, que un recibo de un municipio no se anuncie como nacional, que un fallo TLS no deje el manifiesto anterior como actual, y los ocho casos del hash de contenido: coincidencia por bytes y por contenido, rechazo de un byte distinto fuera del `timeStamp`, rechazo sin `content_sha256`, rechazo de dos `timeStamp` en la raíz, reconstrucción del SHA256 original y el `-backfill-receipt` aceptado y rechazado.

2.6. Resultados

El recibo registra la recepción de las respuestas originales el 8 de noviembre de 2026 entre las 15:24:23 y las 15:25:01 (hora de Bogotá), en 6,4 a 8,5 s por página. El atributo del servidor usa UTC (tabla de páginas). En la noche del 8 de noviembre (21:38–21:40, hora de Bogotá) se volvieron a descargar las seis páginas: tienen el mismo tamaño y, con el `timeStamp` original reinsertado, reproducen exactamente el SHA256 del recibo. Es decir, son el mismo contenido; solo cambió la hora que escribe el servidor.

Tabla 4: Páginas del recibo (`mgn_wfs_source_receipt.json`).

Página	Bytes	`sha256`	`content_sha256`	`timeStamp` original
`page_0000.gml`	26 416 139	`5d600d25b928...`	`ce7a0d0ca809...`	20:23:33Z
`page_0200.gml`	22 174 892	`6900fa419817...`	`09b5856d94dd...`	20:23:40Z
`page_0400.gml`	25 069 453	`168772c3d7bd...`	`89e051c376c9...`	20:23:47Z
`page_0600.gml`	29 395 276	`d6b3ee0b193b...`	`0c271831db52...`	20:23:54Z
`page_0800.gml`	21 884 852	`2b8496abd8a2...`	`31f8cfda05d0...`	20:24:02Z
`page_1000.gml`	27 119 583	`2eadfebad833...`	`11a39ac93152...`	20:24:09Z

El SHA256 del GeoPackage puede cambiar al escribirlo porque el formato guarda metadatos propios; el manifiesto registra el de la corrida actual y la preparación lo comprueba. Lo que se mantiene estable es el producto: a partir de estas páginas, la preparación produce siempre el mismo GeoJSONL (sección 4).

Tabla 5: Capa resultante según `mgn_download_manifest.json`.

Indicador	Valor
Estado	`complete`, `national_complete = true`, `claims_national_download = true`
Registros	1122 (igual a `numberMatched`), 0 geometrías nulas, todas `MultiPolygon`
Año de versión DANE	954 municipios de 2024 y 168 de 2025 (no se filtra)
CRS	EPSG:4686 declarado por el WFS; coordenadas guardadas en longitud/latitud
Unidad de `MPIO_NAREA`	km^2; cociente atributo/área geodésica: mediana 0,9994, mínimo 0,9984, máximo 1,0209
Capa	`municipio.gpkg`, 100,6 MB
Coincidencia con el recibo	6/6 páginas por contenido (respuestas del 8 de noviembre, 21:38–21:40)

2.7. Limitaciones

- No se descargó el ZIP oficial del MGN ni su `.prj`, así que el CRS del archivo original no está probado (`original_archive_crs_proven = false`). Se usa el `srsName` que declara el WFS; el almacenamiento REST del mismo servicio informa EPSG:3857.
- El elemento del área se llama *Área_departamento* aunque el objeto es un municipio; la unidad se confirma con el cociente geodésico, no con el nombre solo.
- Las páginas GML (unos 152 MB en total) y el GeoPackage (unos 101 MB) están fuera de Git por tamaño. La reproducción requiere conservarlas o descargarlas mediante el WFS con TLS activo y cumplir el recibo; no basta cualquier respuesta nueva si cambió su contenido.

3. Filtrado y validación de municipios PDET

Responsable: Samuel Emperador. Script: `scripts/01_validate_mgn.py`.

3.1. Objetivo

Antes de preparar y cargar los municipios hay que comprobar que la capa del MGN y la lista oficial PDET son coherentes entre sí: que cada uno de los 170 municipios PDET existe en el MGN con un código DIVIPOLA correcto, que ningún código está repetido y que las geometrías sirven para MongoDB. El script solo verifica y reporta; no modifica la capa ni repara geometrías (eso lo hace `02_prepare_municipalities.py`).

3.2. Método

1. Lee el manifiesto de la descarga y se detiene (salida 2) si no acredita la capa nacional: sin manifiesto, `status` distinto de `complete`, capa ausente o SHA256 distinto del declarado.
2. Lee la capa (GeoPackage, EPSG:4686) y la hoja `Hoja1` de `MunicipiosPDET.xlsx`.
3. Normaliza los códigos con `zfill_digits` del contrato: `MPIO_CDPMP` y `Código DANE Municipio` a 5 dígitos (5031 → 05031), `DPTO_CCDGO` a 2.
4. Revisa la capa, la lista y el cruce por código (tabla 6). Cada geometría se clasifica como nula, vacía, no poligonal o inválida; para las inválidas se guarda el motivo de `shapely.explain_validity`.
5. Compara los nombres solo como información, separando las diferencias que son únicamente de tildes.

6. Escribe tres salidas en `outputs/`: `mgn_validation_report.json` (resultado de cada verificación y `status` `valid`, `invalid` o `blocked_no_layer`), `mgn_validation_summary.md` y `pdet_mgn_crosswalk.csv` (una fila por municipio PDET).

Una verificación es *bloqueante* si su falla impide cargar los municipios. Las geometrías inválidas fuera de los PDET y las diferencias de nombre solo generan aviso. El código de salida es 0 si todas las bloqueantes pasan y 1 si alguna falla, así que los scripts siguientes (o quien los corra) pueden leer `status` antes de preparar o cargar.

3.3. Resultados

El reporte vigente fue generado por Miguel el 9 de noviembre de 2026 a las 07:27:37 (hora de Bogotá) sobre la capa de 1122 municipios que produce `01_download_mgn.py`. Reproduce el resultado inicial documentado por Samuel el 8 de noviembre: **`valid`**, 19 de 20 verificaciones en OK y un aviso informativo.

Tabla 6: Verificaciones de `01_validate_mgn.py` sobre la capa real. B: bloqueante.

#	Verificación	B	Resultado
1	Campos obligatorios del MGN (`MPIO_CDPMP`, `MPIO_CNMBR`, `MPIO_NAREA`, `DPTO_CCDGO`)	sí	OK
2	Capa nacional con 1122 municipios	sí	OK: 1122
3	CRS EPSG:4686	sí	OK
4	`MPIO_CDPMP` es DIVIPOLA de 5 dígitos	sí	OK: 0 errores
5	Prefijo de `MPIO_CDPMP` = `DPTO_CCDGO`	sí	OK: 0 errores
6	Sufijo de `MPIO_CDPMP` = `MPIO_CCDGO`	sí	OK: 0 errores
7	Sin códigos repetidos en el MGN	sí	OK: 0
8	`MPIO_NAREA` numérica y positiva	sí	OK
9	Geometrías válidas en el MGN	no	OK: 0 inválidas
10	Vértices dentro de la ventana lon/lat de Colombia	sí	OK
11	Lista PDET con 170 filas	sí	OK: 170
12	Códigos PDET numéricos	sí	OK
13	Sin códigos repetidos en la lista PDET	sí	OK: 0
14	16 subregiones PDET	sí	OK: 16
15	Prefijo del código PDET = departamento del Excel	sí	OK
16	Todos los municipios PDET están en el MGN	sí	OK: 0 faltantes
17	Departamento del Excel = `DPTO_CCDGO` del MGN	sí	OK: 0
18	Geometrías de los 170 PDET válidas	sí	OK: 0 inválidas
19	Nombres iguales entre Excel y MGN	no	Aviso: 15 distintos
20	Cruce completo de 170 municipios PDET	sí	OK: 170

Los 170 municipios PDET cruzan con el MGN: no falta ninguno, no hay códigos repetidos en ninguna de las dos fuentes y los departamentos coinciden. Las 1122 geometrías son `MultiPolygon` válidas, ninguna nula o vacía, y todas caen en la ventana de Colombia (un control para detectar ejes latitud/longitud invertidos, sin certificar por sí solo el CRS). El área MGN de los 170 municipios suma 389 182 km^2. La figura 2 muestra el resultado del cruce.

El único aviso son 15 nombres escritos distinto en el Excel y en el MGN (tabla 7): 9 difieren solo en tildes y 6 usan otra forma oficial del nombre. Ninguno afecta el cruce, que es por código, y la cifra coincide con `name_mismatches` del reporte de preparación. Los documentos de `municipalities` usan el nombre del MGN.

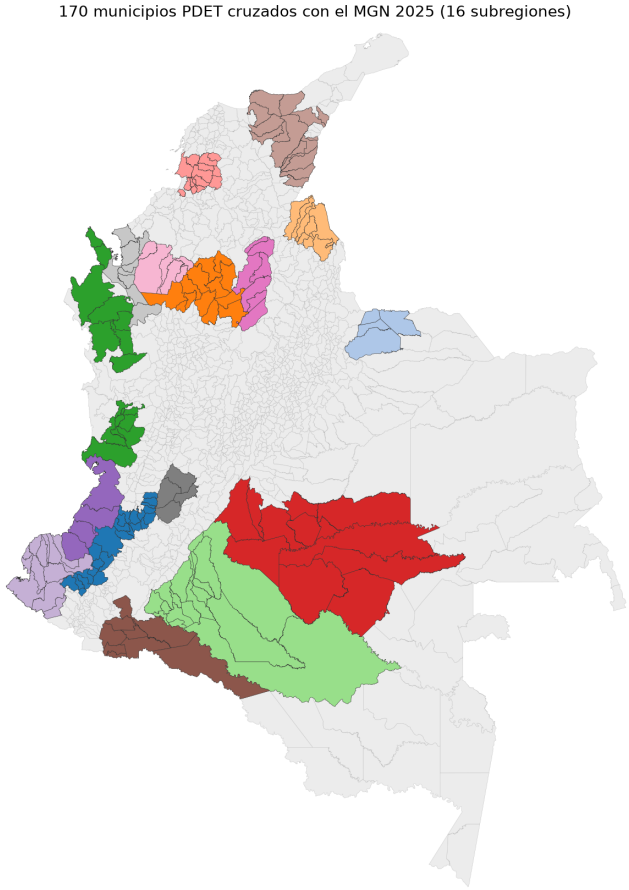


Figura 2: Municipios PDET cruzados con la capa MGN 2025; el color distingue la subregión. Generado con `docs/delivery2/figuras/build_mapa_pdet.py` a partir de `pdet_mgn_crosswalk.csv`.

Tabla 7: Nombres distintos entre `MunicipiosPDET.xlsx` y el MGN 2025.

Código	Excel	MGN	Tipo
19548	PIENDAMÓ	PIENDAMÓ - TUNÍA	forma oficial
50711	VISTA HERMOSA	VISTAHERMOSA	forma oficial
70523	SAN ANTONIO DE PALMITO	PALMITO	forma oficial
70823	TOLÚ VIEJO	SAN JOSÉ DE TOLUVIEJO	forma oficial
19418	LÓPEZ	LÓPEZ DE MICAY	forma oficial
86573	LEGUÍZAMO	PUERTO LEGUÍZAMO	forma oficial
19821	TORIBIO	TORIBÍO	tildes
27099	BOJAYA	BOJAYÁ	tildes
27150	CARMEN DEL DARIEN	CARMEN DEL DARIÉN	tildes
18256	EL PAUJIL	EL PAUJÍL	tildes
70204	COLOSO	COLOSÓ	tildes
52427	MAGÜI	MAGÜÍ	tildes
52835	SAN ANDRES DE TUMACO	SAN ANDRÉS DE TUMACO	tildes
23682	SAN JOSE DE URE	SAN JOSÉ DE URÉ	tildes
05665	SAN PEDRO DE URABA	SAN PEDRO DE URABÁ	tildes

3.4. Pruebas del validador

Como la capa real no tiene errores, cada verificación se probó también con figuras sintéticas (`python scripts/01_validate_mgn.py -self-check`, 19/19): municipio PDET ausente del MGN, código repetido en cada fuente, prefijo de departamento distinto, código no numérico, polígono autointersecado (falla dentro de los PDET y solo avisa fuera de ellos), punto en lugar de polígono, ejes invertidos, CRS EPSG:3857, capa incompleta, campo obligatorio faltante, y manifiesto ausente o bloqueado (salida 2).

3.5. Observaciones

- El cambio de `timeStamp` que bloqueaba la comparación binaria ya se resuelve mediante `content_sha256`, conservando los bytes originales. El manifiesto vigente registra coincidencia por contenido de las seis páginas (sección 2).
- El SHA256 del GeoPackage puede cambiar entre escrituras por sus metadatos; el GeoJSONL preparado sí sale idéntico. Por eso el validador compara la capa con el manifiesto de la misma corrida.

4. Preparación de los documentos de municipios

Responsable: José Medina. Script: `scripts/02_prepare_municipalities.py`.

4.1. Objetivo

Convertir la capa municipal que deja `01_download_mgn.py` en los 170 documentos GeoJSON de los municipios PDET, con la forma exacta que exige el validador de la colección `municipalities` de la Entrega 1, y dejar un reporte que diga qué archivo se produjo y con qué SHA256. La preparación no escribe en MongoDB ni agrega `loaded_at`: esa fecha la pone la carga (sección 5).

4.2. Entradas y salidas

Tabla 8: Contrato de entrada y salida de `02_prepare_municipalities.py`.

Pieza	Rol	Condición
`outputs/mgn_download_manifest.json`	entrada (Derek)	`status = complete`, `claims_national_download = true`, `fixture = false`, `cache_matches_receipt = true`, 1122 registros
`data/raw/mgn2025/wfs/municipio.gpkg`	entrada (Derek)	capa nacional; SHA256 igual al del manifiesto
`resources/MunicipiosPDET.xlsx`, hoja `Hoja1`	entrada	170 filas; `Código DANE Municipio` como número
`data/processed/municipalities.geojsonl`	salida	un documento JSON por línea, ordenado por `_id`, UTF-8 y fin de línea LF
`outputs/municipalities_prepare_report.json`	salida	`role = prepare-report`, `status = ready_for_load`, `claims_full_pdet_set`, `sha256` del GeoJSONL

4.3. Método

1. **Manifiesto verificado.** Se detiene si el manifiesto falta, es un fixture, no está en `complete`, no declara la capa nacional de 1122 registros o si el SHA256 de `municipio.gpkg` no es el que declara. En ese caso el reporte queda en `blocked_no_layer`, `unreadable_layer` o `layer_hash_mismatch`, sin GeoJSONL, y un reporte `ready_for_load` anterior se conserva en `municipalities_prepare_report.history.json`.
2. **Cruce por código DIVIPOLA.** `Código DANE Municipio` se rellena a 5 dígitos (por ejemplo `5031` → `05031`) y se cruza con `MPIO_CDPMP`. El código de departamento del Excel debe coincidir con `DPTO_CCDGO` del MGN. El nombre del municipio sale del MGN; si difiere del Excel, no bloquea y se cuenta en `name_mismatches`.
3. **Nombre del departamento.** Se usa la única columna del MGN cuyos valores coinciden con `Departamento` del Excel (aquí `DPTO_CNMBRE`). Si coinciden dos, se detiene; si no coincide ninguna, se toma del Excel y el reporte lo indica.
4. **Geometría.** Se transforma de EPSG:4686 a EPSG:4326, el CRS que exige el índice `2dsphere`. Una geometría inválida pasa por `make_valid`; no se simplifica. Los anillos exteriores quedan en sentido antihorario. Cada vértice debe caer en la ventana de Colombia (longitud -82 a -66, latitud -5 a 16), y un documento de más de 15 MB se rechaza completo.
5. **Área.** `MPIO_NAREA` se copia a `area_km2` solo si la unidad está declarada; el manifiesto la confirma con el cociente entre el atributo y el área geodésica de cada contorno (mediana 0,9994 sobre los 1122 municipios).
6. **Contrato común.** Cada documento pasa por `document_errors` de `entrega2_contracts.py`, la misma regla que usan la carga y la verificación. Solo con 170 documentos sin rechazos el reporte queda en `ready_for_load` con `claims_full_pdet_set = true`.
7. **Salida reproducible.** Cada línea es `json.dumps` con claves ordenadas, sin espacios y en UTF-8, y el archivo se escribe con fin de línea LF en cualquier sistema operativo. El SHA256 que queda en el reporte es el del archivo ya escrito.

Cada documento tiene `_id`, `name`, `dpto_code`, `dpto_name`, `subregion_id`, `subregion_name`, `area_km2`, `geometry` y `source` (`dataset`, `version`, `file`).

4.4. Pruebas

`python scripts/02_prepare_municipalities.py -self-check` prueba la preparación con capas sintéticas marcadas como fixture. Resultado: **20/20 OK**. Cubre la normalización `5031` → `05031`, el nombre tomado del MGN, la ausencia de `loaded_at`, la conversión de m^2 a km^2 solo con unidad declarada, el descubrimiento de la columna de departamento (y el rechazo si hay dos), la reparación con `make_valid` sin simplificar, la orientación antihoraria, el rechazo de un documento sobre el límite de tamaño, y que un manifiesto fixture, de un solo municipio o sin capa no habilite los 170 municipios.

4.5. Resultados sobre los 170 municipios

Corrida del 9 de noviembre de 2026 sobre la capa reconstruida desde las seis páginas WFS verificadas contra el recibo de Derek (coinciden por contenido: solo cambia el atributo `timeStamp`; ver `entrega2_descarga_y_preparacion.md`): Python 3.11.9, GeoPandas 1.2.0, Shapely 2.2.0 (GEOS 3.14.1) y PyProj 3.7.2.

El SHA256 es el mismo que registró la primera preparación del equipo: la cadena descarga → preparación es reproducible byte a byte desde las mismas páginas WFS, también en Windows.

Tabla 9: Resultado de la preparación.

Indicador	Valor
Estado	`ready_for_load`, `claims_full_pdet_set = true`
Documentos escritos	170 (170 líneas), 0 rechazados, 0 códigos faltantes, 0 repetidos
Geometrías reparadas	0
Nombres distintos entre Excel y MGN	15 (9 solo por tildes); el cruce es por código
Documento más grande	1 341 497 bytes en JSON
Geometrías	170 `MultiPolygon`: 168 de una parte, Tumaco (52835) con 3 y Buenaventura (76109) con 31
SHA256 del GeoJSONL	`612e2544be7ca2c22285aaac647819b910aa779ce61e1e5b872766051101f8ed`

4.6. Diferencia con el archivo de la primera carga

Según la comparación documentada por Miguel, antes de esta corrida la colección se había cargado con un GeoJSONL reconstruido a mano (SHA256 `b229d6a3...`) usando `-allow-input-mismatch`. La comparación por `_id` contra el archivo definitivo se resume a continuación; no se repitió durante esta revisión editorial:

Tabla 10: Archivo reconstruido frente al archivo definitivo, por DIVIPOLA.

Aspecto	Reconstruido (`b229d6a3...`)	Definitivo (`612e2544...`)
Municipios y `_id`	170	los mismos 170
Atributos	idénticos: nombre, departamento, subregión y `area_km2`	
Coordenadas	idénticas en los 170 municipios	
`geometry.type`	168 `Polygon` y 2 `MultiPolygon`: se leyó el GML sin pasar por el GeoPackage	170 `MultiPolygon`: el GeoPackage guarda la capa como `MultiPolygon`
`source.file`	`municipio.gpkg`	`Serv_CapasMGN_2025:Municipio` (campo `filename` del manifiesto)
Fin de línea	CRLF (escrito en Windows)	LF

Las diferencias son de forma, no de contenido geográfico. Para que el SHA256 no dependa del sistema operativo, `write_geojsonl` ahora escribe siempre con fin de línea LF.

5. Carga en MongoDB

Responsable: Miguel Duarte. Script: `scripts/02_load_municipalities.py`.

5.1. Objetivo

La entrada son los 170 documentos producidos por `02_prepare_municipalities.py`. Se cargan en `municipalities`, base `upme_pdet`, con el validador `$jsonSchema` y el índice `2dsphere` activos. La carga debe poder repetirse sin duplicar documentos.

5.2. Entradas y salidas

Tabla 11: Contrato de entrada y salida de `02_load_municipalities.py`.

Pieza	Rol	Condición
`outputs/municipalities_prepare_report.json`	entrada (José)	contrato `entrega2-mgn-v1`, `status = ready_for_load`, `claims_full_pdet_set = true`, `loaded_at_included = false`
`data/processed/municipalities.geojsonl`	entrada (José)	un documento por línea; SHA256 igual al del reporte
colección `municipalities`	salida	170 documentos, `_id` = DIVIPOLA, `loaded_at` como fecha BSON
colección `pdet_subregions`	salida	16 subregiones: `_id`, `name`, `municipality_count`
colección `ingestion_runs`	salida	una fila por corrida, `dataset_id = mgn`
`outputs/municipalities_load_report.json`	salida	`role = load-report`, `status` `loaded`, `verification_failed`, `blocked` u `operational_error`

5.3. Método

1. **Entrada verificada.** Lee el reporte de preparación y se detiene si no está en `ready_for_load` o si el SHA256 del GeoJSONL no es el que José declaró. Con `-allow-input-mismatch` se puede cargar igual, pero el reporte de carga lo deja registrado en `input_matches_prepare_report = false`.
2. **Revisión antes de escribir.** Cada documento pasa por `document_errors` del contrato común (los mismos campos, prefijo de departamento, ventana de Colombia y límite de tamaño que usa la preparación). La carga se detiene si hay un `_id` repetido en la entrada, si alguno ya trae `loaded_at`, si no son 170 o si no son exactamente los códigos del reporte.
3. **Esquema activo antes de insertar.** Importa `SCHEMAS` y `SMALL_INDEXES` de `00_init_db.py` (no los copia) y aplica el validador con `validationAction = error` a `municipalities`, `pdet_subregions` e `ingestion_runs`, y crea `geometry_2dsphere` y `subregion_id_1`. El índice existe *antes* de la inserción, así que MongoDB rechaza en ese momento cualquier geometría que el `2dsphere` no acepte.
4. **Carga idempotente.** El `_id` es el código DIVIPOLA. El script compara cada documento con el que ya está en la base (sin `loaded_at`): si no existe lo inserta, si cambió lo reemplaza y si es idéntico no lo toca. Todo va en un solo `bulk_write` no ordenado. `loaded_at` es la hora UTC de la corrida y solo cambia en los documentos que se insertan o reemplazan.
5. **Subregiones.** Las 16 subregiones salen de los mismos documentos y se escriben con `upsert` por `_id`, con el conteo de municipios de cada una.
6. **Verificación posterior.** Conteo igual a 170, mismos códigos que la entrada, sin `_id` ni pares nombre–departamento repetidos, cero documentos que incumplan el `$jsonSchema`, `loaded_at` presente, índice `2dsphere` sobre `geometry`, y una consulta `$geoIntersects` con un punto interior del primer municipio que lo devuelve usando `IXSCAN` sobre `geometry_2dsphere`.
7. **Trazabilidad.** Cada corrida se registra en `ingestion_runs` (filas leídas, cargadas y rechazadas, inicio, fin y notas), y el detalle queda en `municipalities_load_report.json`.

Si en la base hay un `_id` que ya no está en la entrada, el script lo reporta en `extra_in_db` y no da la carga por buena; no se elimina nada por defecto. La opción de borrado existe para uso explícito, pero no se empleó en la carga definitiva.

5.4. Pruebas

`python scripts/02_load_municipalities.py -self-check` corre la carga completa contra MongoDB real en una base temporal `load_selfcheck_<uuid>` que se borra al terminar, con tres cuadrados sintéticos. Resultado: **16/16 OK**.

Tabla 12: Casos del `-self-check` (MongoDB 7.0.45).

#	Caso comprobado
1	Primera carga: 3 insertados y verificación OK
2–4	Segunda carga idéntica: 0 insertados, 0 reemplazados, 3 sin cambios; siguen 3 documentos y `loaded_at` no cambia
5	Un documento modificado se reemplaza y no se duplica
6–7	`pdet_subregions` con conteos derivados; una fila en `ingestion_runs` por corrida
8–9	Un `_id` repetido en la entrada o un `loaded_at` ya presente detienen la carga
10–11	El validador `$jsonSchema` rechaza un `_id` de 4 dígitos y un documento sin `loaded_at` (código 121)
12	El índice `2dsphere` rechaza un polígono que se auto-intersecta (código 16755)
13	El índice `_id` rechaza insertar otra vez un municipio cargado (código 11000)
14–15	Un `_id` ajeno en la base se reporta y sin `-prune` la carga no se da por buena; con `-prune` se borra
16	La base temporal se borró

5.5. Resultados sobre los 170 municipios

Carga definitiva documentada por Miguel el 9 de noviembre de 2026 (Python 3.11.9, PyMongo 4.19.0, MongoDB 7.0.45 en Docker). El reporte publicado corresponde a la segunda corrida, de 07:30:18 a 07:30:25, hora de Bogotá. La entrada es el GeoJSONL de la preparación (sección 4) con SHA256 `612e2544...`, el mismo del reporte de preparación. La ejecución definitiva fue realizada por Miguel en su computador; el reporte de carga queda con `input_matches_prepare_report = true` y no se usó `-allow-input-mismatch`. La carga se corrió dos veces seguidas sobre la misma entrada:

Tabla 13: Dos cargas definitivas según la documentación de Miguel; el reporte publicado corresponde a la segunda.

Corrida	Leídos	Nuevos	Reempl.	Iguales	Rechazos	Total
1	170	0	170	0	0	170
2	170	0	0	170	0	170

La primera corrida reemplazó los 170 documentos de la carga de prueba anterior (ver la diferencia entre ambos archivos en la sección 4); no insertó ninguno nuevo, porque los 170 `_id` ya existían. La segunda no insertó ni modificó nada: la colección siguió con 170 documentos y 170 `_id` distintos, todos con el mismo `loaded_at` de la primera corrida. Miguel documentó nueve verificaciones posteriores OK en ambas corridas; el JSON versionado permite comprobar directamente la segunda. Además:

- Índices de `municipalities`: `_id_`, `geometry_2dsphere` y `subregion_id_1`. Validador con `validationAction = error` y los diez campos obligatorios del diccionario de datos.
- Geometrías: 170 `MultiPolygon` (168 de una sola parte); el documento más grande, Solano (18756), ocupa 1,75 MB en BSON, lejos del límite de 16 MB.
- `pdet_subregions`: 16 documentos cuyos `municipality_count` suman 170.
- La consulta de Amalfi (05031) del reporte espacial definitivo devuelve solo ese municipio. Su plan ganador usa `geometry_2dsphere`, con 9 claves examinadas y un documento leído (sección 6).

- El verificador de Sebastián (`02_verify_spatial.py`) sobre esta misma base: **9/9 OK**, salida 0 (conteo 170/170, DIVIPOLA, cruce con el Excel sin faltantes ni sobrantes, esquema, 170/170 geometrías válidas, índice, consulta con control negativo y `explain`). La evidencia queda en `outputs/municipalities_spatial_verification_real.json`.

5.6. Comparación de contenido del WFS

La primera carga se hizo con `-allow-input-mismatch`: al volver a descargar, el SHA256 de las páginas GML nunca coincidía con `mgn_wfs_source_receipt.json`, porque el WFS del DANE escribe en el elemento raíz un atributo `timeStamp` con la hora de cada respuesta. Se comprobó que es la única diferencia: con el `timeStamp` original reinsertado, las seis páginas descargadas de nuevo reproducen exactamente el SHA256 del recibo. El recibo ahora guarda, además, un hash de contenido (`content_sha256`, con ese atributo vaciado), y `01_download_mgn.py` acepta una página que coincide por bytes o por contenido (detalle en `entrega2_descarga_y_preparacion.md`). Con eso la cadena completa corrió sin excepciones y la carga ya no necesita `-allow-input-mismatch`.

6. Verificación de la integración y del índice espacial

Responsable: Sebastián Sánchez Vásquez. Script: `scripts/02_verify_spatial.py`.

6.1. Objetivo y método

Verificar mediante lecturas la colección `upme_pdet.municipalities`, sin modificar documentos, índices ni geometrías. El script lee `config.yaml`, admite `MONGO_URI` e importa el esquema BSON de `00_init_db.py` y las constantes del contrato común. Normaliza los códigos del Excel de referencia con ceros iniciales; no corrige los identificadores almacenados.

Se comprueban nueve controles: conexión, cantidad exacta de 170 documentos, formato y unicidad DIVIPOLA, igualdad con el conjunto PDET, esquema y contrato, geometrías, índice espacial, consulta y uso efectivo del índice. El esquema exige, entre otros campos, `loaded_at` como fecha BSON. Las geometrías Polygon/MultiPolygon deben ser no vacías, topológicamente válidas, de área positiva, con anillos explícitamente cerrados y posiciones 2D numéricas finitas en longitud/latitud. Todos los vértices deben estar en la ventana Colombia del contrato; se rechazan ejes invertidos y un CRS explícito alternativo.

Para `$geoIntersects` se toma un punto interior del primer municipio ordenado por `_id`. Los identificadores devueltos deben coincidir exactamente con las intersecciones calculadas por Shapely. El control negativo en longitud 0, latitud 0 debe devolver cero. Se solicita `explain()` con `executionStats`, sin `hint`: el plan ganador debe contener un `IXSCAN` del índice espacial, con ejecución exitosa, claves examinadas y número de resultados coherente. Un índice presente solo en planes rechazados no aprueba este control.

6.2. Pruebas automatizadas existentes

La suite contiene **23 pruebas: 11 unitarias y 12 de integración**. El registro conservado en `docs/delivery2/spatial_test_results.txt` muestra 23/23 aprobadas con MongoDB 7.0.45 en una ejecución anterior. Cubre datos sintéticos válidos, cantidades 0/169/171, identificadores y conjuntos incorrectos, esquema BSON, geometrías inválidas, huecos y MultiPolygon, índice ausente con `COLLSCAN`, resultados espaciales positivos y negativos, planes rechazados y códigos de salida. Dos casos inyectan respuestas incorrectas para comprobar su rechazo. Cada prueba de integración crea una base con UUID y limpia solo sus recursos.

El registro del self-check local del cargador conserva otro resultado histórico de 16/16. La evidencia sintética positiva registra 7 claves examinadas y un documento devuelto. **Estas pruebas no se volvieron a ejecutar durante esta actualización documental** ni sustituyen la verificación definitiva sobre municipios reales.

6.3. Evidencia definitiva: entorno de Miguel

El PR 8 incorporó la cadena definitiva y el reporte completo del verificador. La ejecución fue realizada **por Miguel en su computador**, el 9 de noviembre de 2026 a las 07:30 de Bogotá, después de la carga descrita en la sección 5. No se presenta como una ejecución en el computador de Sebastián. La evidencia está en `outputs/municipalities_spatial_verification_real.json`.

Se contrastaron los reportes de descarga, validación, preparación, carga y verificación: capa nacional de 1122 municipios, cruce de los 170 PDET, preparación `ready_for_load`, carga `loaded` y `input_matches_prepare_report: true`. El SHA256 del GeoJSONL preparado coincide con el de la entrada cargada:

```
612e2544be7ca2c22285aaac647819b910aa779ce61e1e5b872766051101f8ed
```

La segunda carga conserva 170 documentos sin cambios, sin rechazados ni borrados. La preparación definitiva conserva 170 MultiPolygon (sección 4). El reporte espacial registra `success: true` y **nueve controles OK**, coherentes con la salida 0 documentada por Miguel y definida por la CLI.

Tabla 14: Resultados definitivos del verificador independiente, ejecutado por Miguel.

Control	Resultado
Conexión	OK
Cantidad	170/170
DIVIPOLA	Identificadores únicos de cinco dígitos ASCII
Cruce con el Excel	0 faltantes y 0 sobrantes
Esquema y contrato	0 errores
Geometrías	170/170 válidas
Índice	`geometry_2dsphere` visible sobre `geometry`
Consulta	Resultado esperado y obtenido: `05031`; negativo: 0
`explain()`	Plan ganador con `IXSCAN`, ejecución exitosa

6.4. Consulta espacial y ejecución del índice

La consulta registrada usa un punto interior de Amalfi (05031):

```
{geometry: {$geoIntersects: {$geometry: {
  type: "Point",
  coordinates: [-74.9514735167314, 7.020482489999999]
}}}}
```

Devuelve únicamente `05031`, igual al resultado esperado por Shapely. El control negativo `[0.0, 0.0]` devuelve cero coincidencias. El plan ganador es `FETCH` → `IXSCAN`, con `indexName: geometry_2dsphere` y clave espacial `geometry: 2dsphere`. Las estadísticas del JSON registran **9 claves examinadas, 1 documento examinado y 1 resultado**, con `executionSuccess: true`. Esto demuestra uso efectivo del índice en esta consulta; las 7 claves del ejemplo sintético no son la estadística de este JSON definitivo.

6.5. Trazabilidad y antecedente local

La diferencia del WFS quedó resuelta mediante el hash de contenido del recibo: se omite únicamente el valor de `timeStamp` de la raíz, sin modificar los archivos fuente ni los demás bytes. El manifiesto definitivo registra coincidencia por contenido de las seis páginas. Las diferencias del JSONL de la carga anterior (`b229d6a3...`) se explican en la sección 4: representación Polygon/MultiPolygon, `source.file` y CRLF/LF. El producto definitivo recupera el hash `612e2544...` y ya coincide con la preparación; no requiere la excepción de hash anterior.

Como antecedente separado, `docs/delivery2/spatial_local_evidence.json` conserva la base anterior de Sebastián con 0/170 y 2/9 controles aprobados. Ese resultado no describe el estado definitivo del equipo. La evidencia sintética también se conserva aparte. Esta actualización revisó los reportes y su coherencia sin repetir la carga ni sobrescribir la evidencia definitiva con una comprobación local.

6.6. Reproducción y límites

Para otra comprobación, usar un destino nuevo y preservar el reporte definitivo:

```
.venv/bin/python -m unittest discover -s tests -v
.venv/bin/python scripts/02_verify_spatial.py \
  --report outputs/municipalities_spatial_verification_local.json
echo $?
```

El código 0 exige todos los controles OK; 1 indica fallos o pendientes y 2 un error operativo. La reproducción completa necesita los datos pesados y los reportes correspondientes, según el flujo oficial del equipo.

Se revisan estructura y topología en todos los documentos, pero la consulta es una muestra determinista con control negativo, no una certificación de ausencia de huecos o solapamientos entre municipios. Shapely evalúa en el plano y MongoDB sobre la esfera; casos próximos a fronteras pueden requerir revisión. Los rangos no prueban por sí solos el CRS original ni la autenticidad DANE, que dependen de la trazabilidad de descarga y preparación. En este Mac se contrastaron los reportes publicados, sin recalcular los hashes de datos ausentes ni certificar una carga local. La ejecución requiere una colección estable, sin escrituras concurrentes; el plan corresponde a MongoDB 7.0.45 del entorno de Miguel. La guía ampliada está en `docs/delivery2/entrega2_verificacion_espacial.md`.

7. Cierre y entrega

El informe presenta evidencia de adquisición y validación, integridad del GeoJSON e implementación del índice espacial; el plan y el modelo E–R/diccionario se resumen en la sección 1. La cadena completa se reproduce siguiendo la tabla 1 y los comandos del README, con datos y recibos consistentes. La verificación debe usar un archivo de salida nuevo para preservar el reporte definitivo de Miguel.

El PDF se versiona con sus fuentes. El enunciado exige además su envío a BrightSpace antes de las 14:00 de la semana 2, contribuciones de todos los integrantes y defensa del trabajo. Esta revisión no realiza ese envío ni acredita una aprobación individual o una defensa. No se asigna una calificación a los resultados documentados.

Fuentes del curso consultadas: enunciado del proyecto y rúbrica vigente de la semana 2. La fuente administrativa es el WFS MGN 2025 del DANE, identificado en el recibo y en `config.yaml`.